

Claude Code Python

Terminalowy asystent AI do programowania

Zastosowanie Metod Sztucznej Inteligencji

31 stycznia 2026

O projekcie

Cel

Terminalowy asystent AI wykorzystujący Anthropic API do autonomicznego programowania — czyta, edytuje pliki i wykonuje komendy systemowe.

Kluczowe funkcje:

-  Konwersacja z modelem Claude
-  Tool calling (6 narzędzi)
-  Extended thinking (4 poziomy)
-  System 4 agentów
-  Tryb REPL + CLI

Technologie

- **Python 3.8+**
- **anthropic** — SDK
- **Textual** — TUI
- **httpx** — streaming
- **Flask** — OAuth proxy

API

/v1/messages

Model: claude-sonnet-4-5

Auth: API Key / OAuth PKCE

Czat z modelem i edycja plików

```
claude-code-py
Claude Code Python - Sonnet 4.5
Extended Thinking Enabled

Model: claude-sonnet-4-5
Tools enabled (6 tools)

You: Wyjaśnij wzorzec Observer

Thinking...
Thinking complete (1s, 60 tokens)
Claude: Wzorzec Observer to
behavioralny wzorzec projektowy,
w którym obiekt utrzymuje listę
obserwatorów i powiadamia ich
o każdej zmianie stanu.
```

```
claude-code-py
You: Napraw błąd w calculate_total()

Thinking complete (1s, 68 tokens)
Executing tool:read_file
Input: src/billing.py
Tool result (success)
Executing tool:edit_file
Input: linia 42
Tool result (success)
Claude: Brakowało sprawdzenia
czy discount nie jest None.
```

- Pełnoekranowy TUI (Textual)
- Streaming w czasie rzeczywistym
- Widoczny proces myślenia

- Model sam czyta, szuka, edytuje
- 6 narzędzi: Bash, Read, Write, Edit, Grep, Glob
- Pętla do 5 rund automatycznie

Czytanie i edycja plików

claude-code-py

You: widzisz wierszyk?
zmień wers 4 i napisz na nowo
drugą strofę

Thinking complete (1s, 56)

Executing tool:bash
Get-ChildItem

Tool result (success)

Thinking complete (1s, 38)

Executing tool:read_file
wierszyk.txt

Tool result (success)

Thinking complete (2s, 102)

Executing tool:write_file

Tool result (success)

claude-code-py

Claude: Gotowe! Zmieniłem
wers 4 i napisałem nową strofę:

*Na biurku siedzi rudy kot,
Patrzy w ekran uważnie.
Łapką dotyka klawiatury,
Kod pisze nocą dzielnie.*

*Gdy commit nagle się psuje,
Mruczy do monitora.
Refactor robi z zapalem,
I merge'uje do rana.*

- Jedno polecenie → 3 narzędzia
- Model sam znajduje, czyta, edytuje
- Kreatywna modyfikacja treści

Autonomiczne debugowanie

```
claude-code-py
You: Uruchom testy i napraw te które nie przechodzą

Thinking complete (1s, 85 tokens)
Executing tool:bash
Input: pytest tests/ -v
✓ Tool result (success):
FAILED tests/test_auth.py::test_token_refresh
Executing tool:read_file
✓ Tool result (success)
Executing tool:edit_file
✓ Tool result (success)
Executing tool:bash
Input: pytest tests/test_auth.py -v
✓ Tool result (success):
PASSED all tests
Claude: Test naprawiony -- mock nie uwzględniał nowego parametru.
```

- Cykl: uruchom → przeanalizuj → napraw → zweryfikuj
- Cross-platform: Bash / PowerShell / CMD

- Wiele narzędzi w jednej rundzie
- Automatyczna iteracja do sukcesu

Extended Thinking

claude-code-py

You: think deep: wzorce
projektowe w tym projekcie?

Thinking level: DEEP (20,000)

Thinking complete (2s, 120)

Executing tool: bash

Tool result (success)

Thinking complete (3s, 176)

Executing tool: read_file

Tool result (success)

...12 rund narzędzi...

Claude: Factory, Registry,
Chain of Responsibility,
Strategy, Pipeline, DI...

Dynamiczny limit rund narzędzi:
BASIC 5, STANDARD 8, DEEP 12, ULTRA 15.
Graceful fallback przy limicie.

Wyszczególnieni agenci

claude-code-py

You: Zrób code review tego kodu

Thinking...

Thinking complete (3s, 420 tokens)

Executing tool: read_file

Tool result (success)

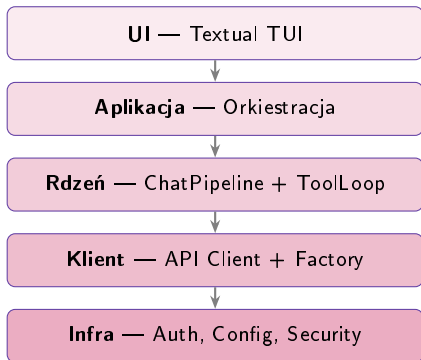
Claude: Przegląd kodu:

! SQL injection (linia 23)

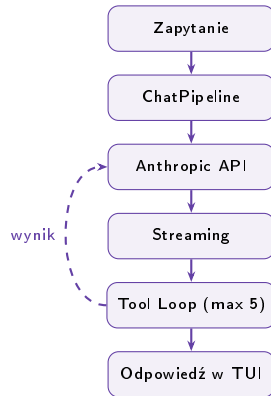
! N+1 query (linia 45)

Spójne nazewnictwo

Router wewnętrznie wybiera agenta
(keyword matching + confidence).
UI pozostaje przezroczyste.



Przepływ danych



Zrealizowane funkcje

- Pełnoekranowy TUI ze streamingiem
- Integracja z Anthropic API
- Tool calling — 6 narzędzi
- Extended thinking — 4 poziomy
- System 4 agentów
- Dual auth (OAuth + API key)
- Tryb interaktywny + CLI

Możliwe rozszerzenia

- Obsługa wielu modeli (Gemini, GPT)
- Persistent historia konwersacji
- Wtyczki / własne narzędzia
- Integracja z Git
- Web UI

Dziękuję za uwagę